MESA SCHOOL OF BUSINESS

Data Foundations · Pre-Read

Session 6: Zepto

Big Mastery Question: Our 10-minute promise breaks somewhere. Which dark store, which hour, which step?

Before you come to class

Read this once, slowly. This session is about dates and timestamps — which behave differently from ordinary numbers and text in ways that trip almost everyone up the first time. Once the core idea in Concept 1 clicks, everything else in this document falls into place quickly. Every table below shows real column letters and row numbers, as always.

The company: Zepto promises grocery delivery in 10 minutes. Every order logs FOUR timestamps — when it was placed, picked, dispatched, and delivered — across many dark stores (mini warehouses) citywide. Your job this session: work out exactly where the 10-minute promise is breaking — which store, which hour of the day, which step of the journey — and that means doing real arithmetic on dates and times.

0. The Dataset You'll Be Working With

Zepto's order-timestamps table has 9 columns (order_id, order_placed_ts, picked_ts, dispatched_ts, delivered_ts, dark_store_id, city, items_count, order_value). Here's a 6-column, 6-row slice:

	A	B	C	E	F	G
1	order_id	order_placed_ts	picked_ts	delivered_ts	dark_store_id	city
2	Z9001	2026-07-14 18:32:00	2026-07-14 18:35:00	2026-07-14 18:44:00	DS12	Bengaluru
3	Z9002	2026-07-14 19:05:00	2026-07-14 19:07:00	2026-07-14 19:21:00	DS12	Bengaluru
4	Z9003	2026-07-14 20:15:00	2026-07-14 20:19:00	2026-07-14 20:33:00	DS07	Mumbai
5	Z9004	2026-07-15 08:02:00	2026-07-15 08:05:00	2026-07-15 08:12:00	DS12	Bengaluru
6	Z9005	2026-07-15 09:45:00	2026-07-15 09:50:00	2026-07-15 10:03:00	DS07	Mumbai
7	Z9006	2026-07-15 21:10:00	2026-07-15 21:14:00	2026-07-15 21:29:00	DS12	Bengaluru

Column A = order_id, B = order_placed_ts, C = picked_ts, E = delivered_ts, F = dark_store_id, G = city. (D = dispatched_ts, H = items_count, I = order_value, not shown.)

1. Dates Are Secretly Numbers

Real world reference:

Imagine every single day since a fixed starting point has a ticket number — January 1st of some very old year is ticket #1, the next day is #2, and so on, forever. A date on your screen LOOKS like text, but underneath, Sheets is really just storing that ticket number. Formatting is just the costume the number wears.

In Zepto's world:

This one idea is the reason you can do math on dates at all. "Delivered time minus placed time" only works because both are secretly just numbers underneath — subtracting them gives you another number, which Sheets can then translate into minutes. Without this, none of the duration formulas later in this document would make any sense.

See it for yourself:

The date 14 July 2026 is, underneath its pretty formatting, simply the number 46217 — counting the days since Google Sheets' starting point (30 December 1899). Change a cell's format from "Date" to "Number" and you'll see this exact number appear.

	B	B
1	**Displayed as a date**	**The number underneath**
2	14/07/2026	46217

Same cell, same value — just two different costumes. Timestamps work the same way, with the time-of-day stored as a decimal fraction of that day.

2. DATE & DATEVALUE

Real world reference:

DATE is like assembling a date from three separate LEGO pieces — a year piece, a month piece, a day piece — snapped together into one real date. DATEVALUE is different: it takes a date that's ALREADY written out as plain text (often from an import or a form) and converts it into a real, usable date.

In Zepto's world:

Sometimes a report export gives you year, month and day as three separate columns instead of one clean date — DATE stitches them back together. Other times a CSV import leaves dates looking correct but secretly stored as TEXT — DATEVALUE fixes that so date math and sorting work properly again.

Syntax:

```
=DATE(year, month, day) =DATEVALUE(date_text)  Example: =DATE(2026,7,14) Example: =DATEVALUE("2026-07-14")
```

See it in the table:

	K	L	M	N
1	**Year**	**Month**	**Day**	**DATE(...) result**
2	2026	7	14	=DATE(K2,L2,M2) → 14/07/2026

3. YEAR, MONTH & DAY

Real world reference:

The reverse of DATE — instead of assembling LEGO pieces into a date, these three pull the pieces back apart, handing you just the year, just the month number, or just the day number.

In Zepto's world:

You want to check whether performance changed between 14 July and 15 July. DAY pulls out just the day number from order_placed_ts, so you can group and compare orders by which calendar day they happened.

Syntax:

```
=YEAR(date)   =MONTH(date)   =DAY(date)  Example: =YEAR(B2)   =MONTH(B2)   =DAY(B2)
```

See it in the table:

	A	B	K	L	M
1	order_id	order_placed_ts	YEAR	MONTH	DAY
2	Z9001	2026-07-14 18:32:00	=YEAR(B2) → 2026	=MONTH(B2) → 7	=DAY(B2) → 14
5	Z9004	2026-07-15 08:02:00	=YEAR(B5) → 2026	=MONTH(B5) → 7	=DAY(B5) → 15

4. HOUR

Real world reference:

If a full timestamp is a complete address, HOUR is like extracting just the house number — the one piece of the time-of-day you actually care about.

In Zepto's world:

This is the single most important formula for your mastery question. "Which hour is the promise breaking" needs you to strip every timestamp down to just its hour, so you can group hundreds of orders into "6 PM orders," "7 PM orders," and so on — and see which hour has the worst breach rate.

Syntax:

```
=HOUR(timestamp)  Example: =HOUR(B2)
```

See it in the table:

	A	B	K
1	order_id	order_placed_ts	Hour
2	Z9001	2026-07-14 18:32:00	=HOUR(B2) → 18
4	Z9003	2026-07-14 20:15:00	=HOUR(B4) → 20
7	Z9006	2026-07-15 21:10:00	=HOUR(B7) → 21

HOUR uses the 24-hour clock — 18 means 6 PM, 21 means 9 PM. Once every order has an hour, COUNTIFS from Session 3 can count breaches per hour instantly.

5. WEEKDAY

Real world reference:

Like checking a calendar to see if today is a weekday or the weekend — WEEKDAY turns any date into a simple number representing which day of the week it fell on.

In Zepto's world:

Weekend grocery runs behave very differently from weekday ones — more orders, busier stores, more strain on the 10-minute promise. WEEKDAY lets you tag every order as falling on a particular day, so you can compare weekday performance against weekend performance.

Syntax:

```
=WEEKDAY(date)  By default: 1 = Sunday, 2 = Monday, 3 = Tuesday ... 7 = Saturday
Example: =WEEKDAY(B2)
```

See it in the table:

	A	B	K
1	order_id	order_placed_ts	WEEKDAY
2	Z9001	2026-07-14 18:32:00	=WEEKDAY(B2) → 3 (Tuesday)
5	Z9004	2026-07-15 08:02:00	=WEEKDAY(B5) → 4 (Wednesday)

6. EOMONTH

Real world reference:

Imagine every date on a calendar has a little tag pointing to "the last day of THIS month" — no matter if today is the 2nd or the 28th, the tag always points to the same place. EOMONTH ("End Of Month") does exactly that.

In Zepto's world:

When you eventually compare performance across many months, you need every order neatly bucketed into ITS month — and EOMONTH is the standard trick for that: it turns any date, no matter the exact day, into one consistent "month label" (the month's last date) that's perfect for grouping in a pivot table.

Syntax:

```
=EOMONTH(start_date, months_offset)  Example: =EOMONTH(B2, 0)   ← 0 means "this same month"
```

See it in the table:

	A	B	K
1	order_id	order_placed_ts	Month Bucket
2	Z9001	2026-07-14 18:32:00	=EOMONTH(B2,0) → 31/07/2026
5	Z9004	2026-07-15 08:02:00	=EOMONTH(B5,0) → 31/07/2026

Both orders happened on different days, but both get the SAME month bucket — 31 July 2026 — which makes grouping by month in a pivot table completely reliable.

7. TEXT

Real world reference:

TEXT is a costume-changer for numbers and dates — it takes the raw underlying value and dresses it up in whatever exact written style you ask for, and the result becomes plain readable text.

In Zepto's world:

A raw timestamp like "2026-07-14 18:32:00" is technically correct but clunky on a dashboard. TEXT can turn it into something a founder actually wants to read at a glance — like "Jul 2026" for a monthly summary, or "14-Jul, 6:32 PM" for a detailed log.

Syntax:

```
=TEXT(value, format_pattern)  Example: =TEXT(B2, "mmm yyyy") Example: =TEXT(B2, "dd-mmm, h:mm AM/PM")
```

See it in the table:

	A	B	K
1	order_id	order_placed_ts	TEXT(B2,"mmm yyyy")
2	Z9001	2026-07-14 18:32:00	=TEXT(B2,"mmm yyyy") → Jul 2026

8. Timestamp Subtraction (Duration)

Real world reference:

If a race starts at a certain point and ends at another, subtracting the two tells you how long the race took. Because dates and times are secretly just numbers (Concept 1), subtracting one timestamp from another works exactly the same way — the result is simply "how much time passed."

In Zepto's world:

This is the formula that answers your entire mastery question. Subtracting order_placed_ts from delivered_ts tells you exactly how long each delivery took — but the raw result comes out as a fraction of a DAY, not minutes, so you multiply by 24 (hours in a day) and then 60 (minutes in an hour) to get a number you can actually read and compare against the 10-minute promise.

Syntax:

```
=(end_time - start_time) * 24 * 60     ← converts the day-fraction into minutes
Example: =(E2-B2)*24*60
```

See it in the table:

	A	B	E	K	L
1	order_id	order_placed_ts	delivered_ts	Delivery Minutes	Breached 10-min?
2	Z9001	2026-07-14 18:32:00	2026-07-14 18:44:00	=(E2-B2)*24*60 → 12	YES
5	Z9004	2026-07-15 08:02:00	2026-07-15 08:12:00	=(E5-B5)*24*60 → 10	NO
7	Z9006	2026-07-15 21:10:00	2026-07-15 21:29:00	=(E7-B7)*24*60 → 19	YES

Z9001 and Z9006 both breached the 10-minute promise; Z9004 landed exactly on the line. Combine this with HOUR and dark_store_id, and you can see precisely where and when the promise is breaking.

Coming to class

You now know how to pull apart, rebuild, and do real arithmetic on dates and timestamps. In class, we'll calculate delivery duration for every order, bucket it by store and by hour, and find exactly where Zepto's 10-minute promise is breaking down.